Production Orders API — API Specification Design (API仕様設計)

> **Revision of 2026-10-24 (WI-017).** This file supersedes `001_DD-API_製造指示登録・編集.md`, which stays unchanged as the earlier approved revision. Change: the production-order notes limit is 1000 characters (REQ-090, supersedes the 500 of REQ-016); see `work-items/WI-017/`.

001_DD-API — supports 001_DD; handlers designed in 001_DD-FN; requirements REQ-010–REQ-018.

Document control (改版履歴)

Field	Value
Document ID	001_DD-API
System name	ProductionManagementAI
Subsystem name	Production orders
Work item	WI-002
Created by	Claude (for ThanhTN)
Created date	2026-09-18
Last updated by	Claude (for ThongTM)
Last updated date	2026-10-24

Version	Date	Author	Revision content
1	2026-09-18	Claude (for ThanhTN)	Initial creation
2	2026-09-22	Claude (for ThanhTN)	Reviewed for WI-004 REQ-033: no contract change. The completion time is not added to `ProductionOrderResponse` (Screen A does not show it) and is not accepted in any request; the request DTOs have no such property, so a client-sent field of that name binds to nothing
3	2026-09-23	Claude (for ThanhTN)	WI-005: the UI is Japanese. The example product name is a Japanese automobile part (WI-005 DEC-002). Problem Details `title` texts stay English: they are API text for developers, never shown in the UI, which renders the message ID from `code`. No field, rule, API or processing changes
4	2026-10-24	Claude (for ThongTM)	WI-017 (REQ-090–REQ-092): notes limit 500 → 1000 characters. `notes` maximum length 1000 in `ProductionOrderResponse`, the POST body and the PUT body. No other contract change; MSG-E006 keeps its ID (DEC-004). Dated file name per WI-017 DEC-002

Overview and operation catalog

Field	Value
API / module name	Production orders API
Overview	JSON endpoints behind SCR-001. `GET /api/products` is expected to be reused by Screen B's filters, which is why this is a separate document

No	Endpoint / method	Handler	Purpose	Notes
1	`GET /api/products`	`ProductsController.List` → `ProductionOrderService.ListProductsAsync`	Product options	
2	`POST /api/production-orders`	`ProductionOrdersController.Create` → `CreateAsync`	Create an order	
3	`GET /api/production-orders/{id}`	`ProductionOrdersController.Get` → `GetAsync`	Load an order	
4	`PUT /api/production-orders/{id}`	`ProductionOrdersController.Update` → `UpdateAsync`	Update an order	Full replacement of the editable fields

Common to every endpoint:

- **Auth:** cookie session (0002_ADR) plus policy `ProductionOrderEditor` = role `Admin` or `Operator` (DEC-001). No session → 401 with an empty body. Session without the role → 403 with an empty body. This is the existing WI-001 cookie-event behavior, kept unchanged.
- **Content type:** request bodies must be `application/json`, otherwise 415 (DEC-020). Responses are `application/json`; errors are `application/problem+json` (RFC 9457).
- **JSON:** camelCase; dates as `YYYY-MM-DD`; timestamps ISO 8601 UTC with `Z`; enums as strings.
- **Error body:**

```
{
  "type": "urn:pmai:problem:validation",
  "title": "One or more fields are invalid.",
  "status": 400,
  "code": "VALIDATION",
  "errors": { "quantity": ["MSG-E003"], "dueDate": ["MSG-E005"] },
  "traceId": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01"
}
```

`errors` values are message IDs from 001_DD's catalog; the client maps them to text, so wording lives in one place. Stack traces and exception messages are never included. JSON binding failures (e.g. `"quantity": "abc"`, a malformed date) are mapped to the same shape by a custom `InvalidModelStateResponseFactory`: framework keys like `$.quantity` become `quantity`, and the framework messages become the field's message ID. Framework text is never returned.

System-wide API registry entries

ID	Endpoint	Overview	Notes
API-PRD-01	`GET /api/products`	List products	
API-PO-01	`POST /api/production-orders`	Create production order	
API-PO-02	`GET /api/production-orders/{id}`	Get production order	
API-PO-03	`PUT /api/production-orders/{id}`	Update production order	

No project-level API registry exists yet; these IDs start it.

Shared response object: `ProductionOrderResponse`

No	Name	Variable name	Type	Length	Required	Repeats (array)	Value mapping	Example	Description	Notes
1	ID	`id`	string (uuid)	36	yes	no	—	`3f2c…`	`production_orders.id`	
2	Order number	`orderNumber`	string	13	yes	no	—	`PO-2026-00042`	Generated column	
3	Product ID	`productId`	string (uuid)	36	yes	no	—			
4	Quantity	`quantity`	integer	—	yes	no	—	`250`		
5	Due date	`dueDate`	string (date)	10	yes	no	—	`2026-10-01`		
6	Status	`status`	string	—	yes	no	Enum name	`InProgress`		
7	Allowed next statuses	`allowedNextStatuses`	string[]	—	yes	yes	From `status.AllowedNext()`	`["Completed","Cancelled"]`	Empty for terminal statuses	Drives M-02

No	Name	Variable name	Type	Length	Required	Repeats (array)	Value mapping	Example	Description	Notes
8	Product/quantity editable	`isProductQuantityEditable`	boolean	—	yes	no	`status == Draft`	`false`		Drives the lock UI (REQ-018)
9	Notes	`notes`	string | null	1000	yes	no	—	`null`		
10	Created at	`createdAt`	string (date-time)	—	yes	no	UTC	`2026-09-18T01:02:03Z`		
11	Updated at	`updatedAt`	string (date-time)	—	yes	no	UTC			
12	Version	`version`	integer (uint32)	—	yes	no	`xmin`	`81234`	Echo back on PUT	DEC-014

Endpoint / method design

1. GET /api/products

Field	Value
Description	All products, ordered by `sku` ascending
Return type	`ProductResponse[]`
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments: none.

Processing overview: no-tracking projection `SELECT id, sku, name FROM products ORDER BY sku`. No paging, since the list is 30 rows (DEC-019). Add paging or search when Screen B needs it.

Processing flow

Step	Description	Calls
1	Authorize	policy `ProductionOrderEditor`
2	Query and project	`IProductionOrderRepository.ListProductsAsync`
3	200	—

Return value

Type	Name	Description
`ProductResponse[]`	body	See response fields

Request fields: none.

Response fields (each array element)

No	Name	Variable name	Type	Length	Required	Repeats (array)	Value mapping	Example	Description	Notes
1	ID	`id`	string (uuid)	36	yes	yes	—			
2	SKU	`sku`	string	50	yes	yes	—	`P-1004`		
3	Name	`name`	string	200	yes	yes	—	ドライブシャフト		

Error codes

Code	Meaning	HTTP status
—	Not signed in	401
—	Missing role	403
`MSG-E013`	Unexpected failure	500

2. POST /api/production-orders

Field	Value
Description	Create a production order in `Draft` with the next `PO-YYYY-NNNNN` number
Return type	`ProductionOrderResponse`

Field	Value
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments: request body below.

Processing overview: server side of P-02. Method-level design is in 001_DD-FN §3 `CreateAsync`; the controller boundary is in 001_DD-SPD §5.

Processing flow

Step	Description	Calls
1	Authorize; require the JSON content type	policy; `[Consumes]`
2	Required/shape/range checks; collect all field errors	`ProductionOrder.ValidateQuantity` / `ValidateNotes`
3	Product exists; `dueDate ≥ IPlantClock.Today`	repository; `IPlantClock`
4	Transaction: `IOrderNumberIssuer.NextAsync(IPlantClock.CurrentYear)` → `ProductionOrder.Create` → insert → commit	001_DB counter upsert
5	201, `Location: /api/production-orders/{id}`	—

Return value

Type	Name	Description
`ProductionOrderResponse`	body	The created order

Request fields

No	Name	Variable name	Type	Length	Required	Source	Example	Description	Notes
1	Product ID	`productId`	string (uuid)	36	yes	request body		Must exist	MSG-E001 / MSG-E002
2	Quantity	`quantity`	integer	—	yes	request body	`250`	1–999,999,999	MSG-E003 / MSG-E010. A non-integer JSON value (e.g. `2.5`, `"abc"`) → MSG-E003
3	Due date	`dueDate`	string (date)	10	yes	request body	`2026-10-01`	≥ plant today	MSG-E004 / MSG-E005

No	Name	Variable name	Type	Length	Required	Source	Example	Description	Notes
4	Notes	`notes`	string | null	1000	no	request body		Trimmed; empty → null; ≤ 1000 code points	MSG-E006

Other properties (e.g. `status`, `orderNumber`) are ignored; a new order is always `Draft`.

Response fields: `ProductionOrderResponse` (above).

Error codes

Code	Meaning	HTTP status
`VALIDATION`	One or more field errors (`errors` map)	400
—	Not signed in / missing role	401 / 403
—	Body not `application/json`	415
`MSG-E013`	Counter overflow or unexpected failure	500

3. GET /api/production-orders/{id}

Field	Value
Description	One order by ID
Return type	`ProductionOrderResponse`
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments

No	Type	Name	Description
1	uuid (route, `{id:guid}`)	`id`	A non-GUID value doesn't match the route → 404

Processing overview: no-tracking load and projection, including `xmin` as `version`.

Processing flow

Step	Description	Calls
1	Authorize	policy
2	Load	repository
3	200, or 404 `MSG-E011`	—

Return value

Type	Name	Description
`ProductionOrderResponse`	body	

Request fields: route `id` only.

Response fields: `ProductionOrderResponse`.

Error codes

Code	Meaning	HTTP status
`MSG-E011`	No order with this ID	404
—	Not signed in / missing role	401 / 403

4. PUT /api/production-orders/{id}

Field	Value
Description	Replace the editable fields and optionally change the status, guarded by `version`
Return type	`ProductionOrderResponse`
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments

No	Type	Name	Description
1	uuid (route)	`id`	

Processing overview: server side of P-03, method-level design in 001_DD-FN §4 `UpdateAsync`. The check order is fixed: 400 (shape) → 404 → 409 (version) → 422 (Domain rules) → 400 (data-dependent) → 409 (race at save).

Processing flow

Step	Description	Calls
1	Authorize; JSON content type	policy; `[Consumes]`
2	Shape/required/range checks	Domain validators
3	Load tracked entity → 404	repository
4	`version` ≠ current `xmin` → 409 `MSG-E009`	—
5	`order.Update(...)` → 422 with `code` MSG-E008 or MSG-E007	Domain
6	Changed product exists? Changed due date ≥ plant today? → 400	repository; `IPlantClock`
7	`SaveChanges`; `DbUpdateConcurrencyException` → 409	EF Core
8	200 with a fresh `ProductionOrderResponse`	—

Return value

Type	Name	Description
`ProductionOrderResponse`	body	Saved state, new `version`

Request fields

No	Name	Variable name	Type	Length	Required	Source	Example	Description	Notes
1	Product ID	`productId`	string (uuid)	36	yes	request body		Must equal the saved value unless `Draft`	V-01, V-07
2	Quantity	`quantity`	integer	—	yes	request body		Same as POST; must equal the saved value unless `Draft`	V-02, V-07
3	Due date	`dueDate`	string (date)	10	yes	request body		≥ plant today only if changed	V-03, V-04
4	Status	`status`	string	—	yes	request body	`Completed`	Current status or one of `allowedNextStatuses`; unknown value → 400	V-06
5	Notes	`notes`	string | null	1000	no	request body		Same as POST	V-05
6	Version	`version`	integer (uint32)	—	yes	request body	`81234`	From the last GET/PUT response	V-08

Response fields: `ProductionOrderResponse`.

Error codes

Code	Meaning	HTTP status
`VALIDATION`	Field errors (`errors` map)	400
—	Not signed in / missing role	401 / 403
`MSG-E011`	No such order	404
`MSG-E009`	Stale `version`	409
—	Body not `application/json`	415
`MSG-E007` / `MSG-E008`	Disallowed transition / locked field changed	422
`MSG-E013`	Unexpected failure	500

Problem `type` URIs: `urn:pmai:problem:validation` , `…:not-found` , `…:conflict` , `…:rule-violation` , `…:internal` . The top-level `code` is `VALIDATION` for 400 (details in `errors`) and the 001_DD message ID for every other error (404 `MSG-E011` , 409 `MSG-E009` , 422 `MSG-E007` / `MSG-E008` , 500 `MSG-E013`). 401/403/415 keep their framework bodies.

Unresolved decisions

None. Error-contract choices are recorded as DEC-023 in `work-items/WI-002/decisions.md` .